RegNo: 222008181

UNIVERSITY OF RWANDA

Dept: CSE

On 22nd March 2026

Mobile Application Systems and Design (#Lab 5)

UWIZEYIMANA Gaston
222008181
CST / CSE

On 22nd March 2026

Mobile Application System and Design # Lab 5

1. Which Dependencies did you use and why? Explain why SQLite is necessary to store data locally.

* Run time dependencies (used by the app itself)
  > flutter - Flutter SDK; required to build and run the UI and app logic
  > Cupertino_icons - IOS-style icon font used with material icons in the interface
  > sqflite - plugin that talks to SQLite on the device so posts can be stored in a local database file
  > Path - builds correct file paths (for example joining the app documents directory with the database filename).
  > sqflite_common_ffi - lets the same SQLite code run on windows, Linux, or macOS when developing on a desktop, not only on Android or iOS.
  > intl - formats dates and times when showing when a post was created.
  > google_fonts - loads the custom font (Plus Jakarta Sans) used in the app theme.

* Development dependencies (only for building, testing and tooling)
  > flutter_test : testing framework used for widgets tests
  > flutter_lints : lint rules to keep dart code style consistent
  > flutter_launcher_icons : generates launcher icons from the project image (for example assets/logo.png).

* Why SQLite for local storage?
  The assignment needs posts stored on the device with no internet SQLite gives a single file database with tables and typed columns, so we can insert, query, update, and delete rows reliably. It persists after the app closes and scales better than only using key-values storage or plain files when there are many posts and fields to manage.

2. How do you handle database exceptions?

- Database not initialized: The database is opened through a single accessor that every operation awaits first. If opening or the first query fails, the list screen shows an error and a retry action instead of continuing with null database.

- Insert/update/delete errors: Those calls are wrapped in try/catch. On DatabaseException, the user sees a SnackBar, so they know the save or delete failed; the app does not assume success.

- Invalid or corrupted data: Rows from SQLite are converted into model objects with checks on types and required fields. If a row is wrong or incomplete parsing returns null or the row is skipped, so the list does not crash and the detail view can show that the post could not be read.

3. Explain SQLite in Flutter.

In Flutter SQLite means using using SQLite engine (a small rational database stored as a file on the device) from a Flutter app through a plugin (sqflite). Flutter/Dart does not run SQL inside the dart VM by itself; the plugin calls native SQLite on android and iOS. On desktop, a related setup (Sqflite_Common_ffi) can use SQLite the same way for development.

So in short, SQLite in flutter is an offline, local, structured storage accessed from dart via asnc database APIs.

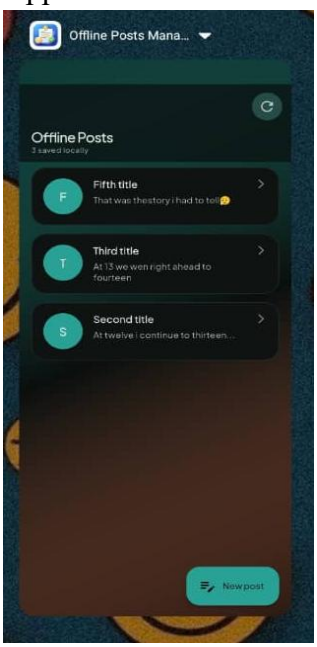
- Database vs table: The database is that whole SQLite file while the table is one named sheet of rows and columns inside it (e.g: posts).

- CRUD: Create for inserting rows, Read for querring rows, Update for changing rows and Delete for removing row(s).

- Asynchronous interactions: plugin methods return Futures, you await them and then refresh the UI, so database work does not freeze the interface.

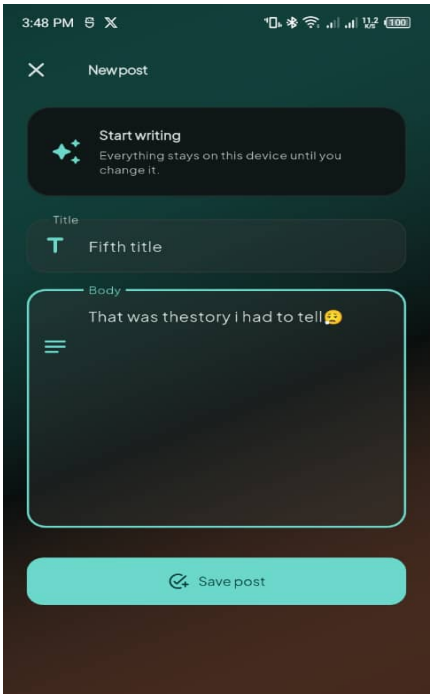
Screenshots:

1. App overview:

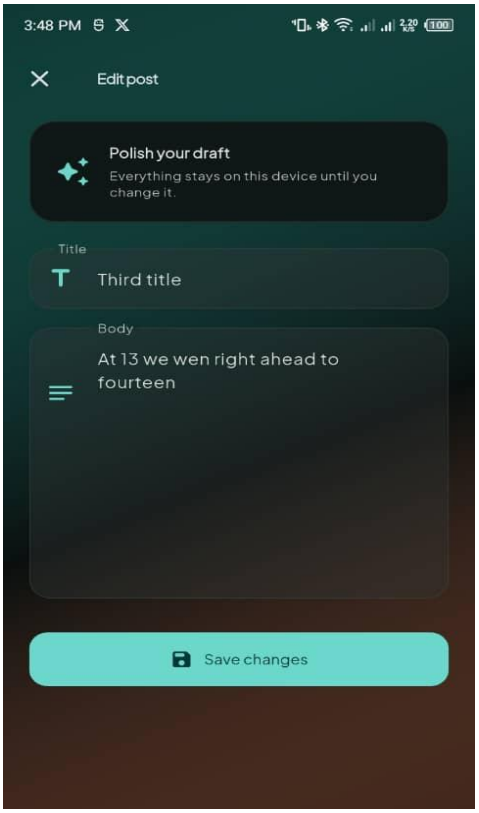


2. All posts view

3. Creating new post

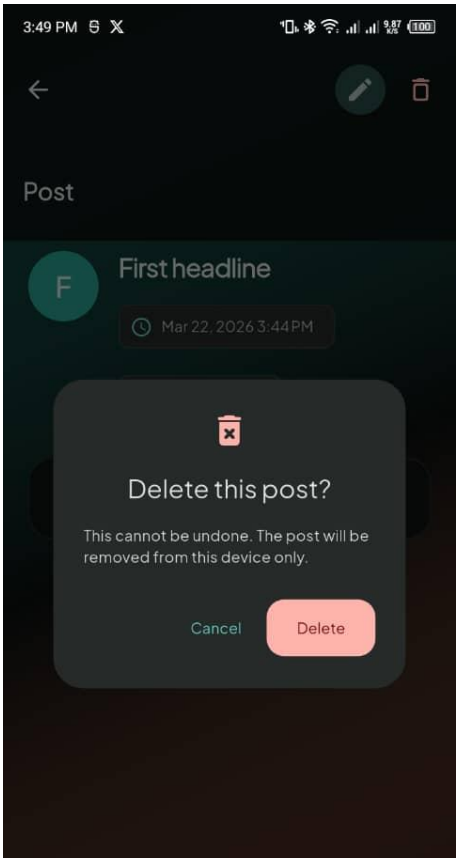
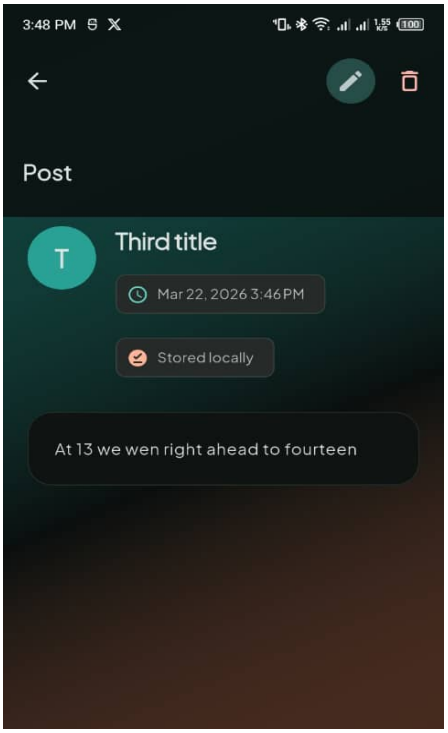


4. Updating the post



5. Post details view

6. Post deletion



7. After post deletion